BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
26 October 2026

Abstract

Agent services need private working state and repeated access to shared repositories, dependencies, and datasets. Reusing file contents leaves the costs of representing edits, querying shared metadata, and serving partial reads. We present BrewFS, an object-backed distributed file system with three independently optional mechanisms. Delta-Based Workspaces retain private namespace records and data-range mappings over a pinned baseline, preserving references to unchanged data. Packed Metadata organizes a published read-only directory version into authenticated pages that co-locate names, common attributes, and data locators. These pages serve supported queries outside the mutable metadata database, targeting stable small-file collections shared by many readers. Variable-Granularity Object Storage separates client object formation from backend encoding and verification. A co-designed RustFS extension accepts coding-block-size hints and persists each object's effective layout for subsequent reads and repair. A native workspace can fork without packing, and a packed version can serve independent read-only access. We describe the representations and update protocols, and characterize 1,138,884 FUSE requests from eight coding-agent runs to establish access patterns for evaluation.

1 Introduction

Agent services run several attempts from the same source revision, dependency tree, or dataset. Each attempt reads shared inputs, makes private edits, and produces artifacts that later attempts may reuse. Coding agents exercise this pattern through repository exploration, builds, and tests [24, 27]. An attempt may also continue on another worker, requiring its saved files to remain accessible after the original container exits. The file system must therefore support both isolated modification and repeated access to shared state throughout execution.

Sharing file contents addresses only part of this cost. Materializing a private namespace for every attempt repeats metadata work even when the payloads are shared. Copy-on-write overlays avoid that initial copy, but their update granularity determines how much inherited state a small edit materializes. OverlayFS and BranchFS establish the utility of private upper layers [13, 22]; the remaining question is how to represent name and byte-range changes while retaining references to the rest of a distributed baseline.

Shared inputs introduce a separate access cost. Consider ML experiments that reuse a fixed training dataset while writing different code, checkpoints, and outputs [3]. For a collection of many small files, repeated enumeration and lookup can place substantial demand on the metadata service used by mutable workspaces. Immutable distribution systems already show how to serve fixed trees through indexed metadata and on-demand access [2, 15, 23]. Applying that approach to a selected dataset requires deciding which records to publish together, how readers locate them, and how much reuse repays preparation of the version.

The bytes returned by those lookups introduce a third cost. A small file read may fetch a larger client object, and the object store may read and verify still larger shard intervals. Small objects reduce the amount fetched by whole-object readers but increase object count and request overhead. Backend encoding and integrity checks impose their own units. Controlling this work requires an interface between file-system object formation and the object store's physical layout.

We present BrewFS, an object-backed distributed file system that separates private workspace state, read-only metadata representation, and payload layout. A shared baseline fixes what a reader inherits; the representations of private changes, metadata, and objects determine the work required to serve that view. BrewFS implements three mechanisms:

- **Delta-Based Workspaces** retain private namespace records and data-range mappings over a pinned immutable baseline. An edit replaces the affected mappings while unchanged ranges retain their references. Deletion and truncation rules prevent discarded state from reappearing (Section 4).

- **Packed Metadata** serves a published read-only directory version through authenticated joint pages that co-locate names, common attributes, and data locators. Supported queries use these pages in object storage and client caches, removing their per-file database access. The representation targets stable small-file collections reused by many readers (Section 5).
- **Variable-Granularity Object Storage** separates client object formation from backend encoding and verification. We co-design the interface with RustFS: BrewFS can attach a coding-block-size hint to a finalized object, and our RustFS extension validates the hint and persists the effective layout for reads and repair (Section 6).

The mechanisms are independently optional. A native-metadata workspace can fork without constructing a pack; a published pack can serve standalone read-only access; an ordinary volume can use the layout interface. This choice matters because their costs differ. Packing introduces a per-version preparation cost, while finer object and verification units trade reduced over-read for more requests and checksums. The appropriate configuration depends on the state being shared and how it is accessed.

We implement these mechanisms and characterize 1,138,884 FUSE requests from eight coding-agent runs. Metadata methods account for 57.3% of the requests, with mixed read/write lengths and concentrated bursts of activity (Figure 1). These observations define access patterns for evaluation. Section 8 sets out the controlled comparisons for mechanism costs, fork, worker handoff, and concurrent execution.

2 Background and Motivation

2.1 Private work and shared inputs

An agent's workspace contains source files, dependencies, generated artifacts, and temporary files. Two attempts starting from the same revision may read the same file and then make incompatible edits. Each must continue to see its own version, including after deletion or truncation. A stable baseline identity and private changes provide this isolation independently of where metadata and bytes reside.

Read-only inputs have a different lifetime from working state. In the machine-learning scenario, an experiment reads a fixed dataset while writing model checkpoints and transformed outputs elsewhere. A directory-based image loader, for example, enumerates a dataset to build a sample list, then opens selected files during loading [21]. Separate experiments repeat this discovery work against the same version, so the namespace of that dataset is read repeatedly while the data itself never changes.

Such a collection permits an organization specialized for reading: its records can be prepared once and reused for the lifetime of a version. Small files make this organization particularly relevant. Every file requires a name, attributes, and a data locator, so metadata work grows with file count even when total payload size stays fixed.

2.2 From file operations to storage work

Reading a file requires path resolution, a data mapping, and access to the referenced bytes. Metadata services batch and cache records [18, 19], while immutable distributions organize them for demand fetching [2, 15]. A file-system call therefore need not cause a remote metadata request. The relevant question is how much database traffic remains under the actual cache and concurrency conditions.

Payload access introduces another translation. Kernel caching and readahead determine which requests reach FUSE; the client then fetches object ranges or complete objects. Encoding and integrity checks may require still larger physical reads [1, 6]. Writes pass through buffering and object formation before publication. FUSE request lengths thus describe neither application syscall sizes nor physical storage units, and a mechanism that changes one layer does not by itself change the others.

2.3 Observed requests in coding-agent runs

Capture. We reanalyzed eight SWE-bench Verified task runs [7]. All used the same harness configured for `deepseek-v4-flash`. Three runs had a broader investigation and testing prompt and a 30-minute budget; all finished earlier. Each base-commit checkout used an instrumented passthrough file system on Ubuntu 24.04 under WSL. Instrumentation recorded requests reaching FUSE handlers.

The capture contains 1,138,884 requests across all eight runs. Counts include all statuses, setup, and teardown. Pooled statistics weight requests equally rather than tasks. The companion workload record retains instrumentation versions, per-task counts, and source-validation details.

Metadata traffic. Six metadata methods (`lookup`, `getattr`, `getxattr`, `readdirplus`, `opendir`, and `releasedir`) accounted for 57.3% of all calls (Figure 1(a)). The Other category contains methods outside these six and read/write. Among successful events, the metadata share was 45.8%, showing the contribution of unsuccessful probes. Across individual tasks, all-status shares ranged from 39.6% in SymPy to 65.4% in Django. Read calls outnumbered writes by 4.87:1 overall; per-task ratios ranged from 3.12:1 to 42.7:1.

Request sizes. All 120,434 reads and 24,712 writes succeeded. Reads ranged from 4 to 128 KiB, with a median of 12 KiB. Writes ranged from one byte to 128 KiB, with a median of 4 KiB. Requests of at most 4 KiB accounted for

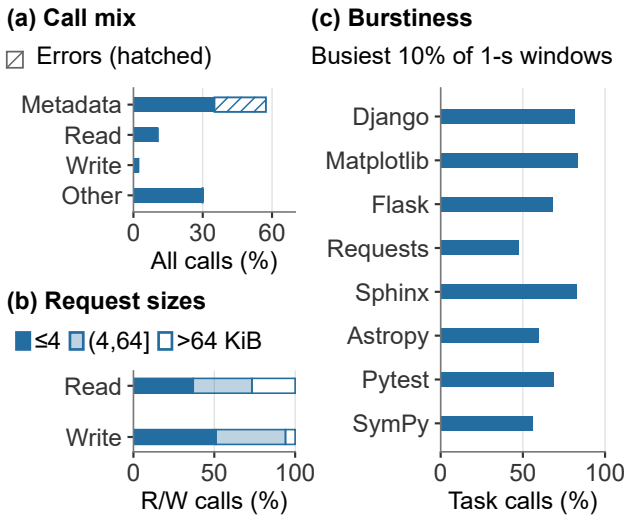


Figure 1: FUSE requests from eight coding-agent runs (1,138,884 total). (a) Operation shares across all statuses; hatching marks errors within each category. (b) Requested lengths in KiB, normalized separately for reads and writes. (c) Each task's request share in its busiest 10% of one-second windows, including idle and final partial windows. Panels (a,b) pool requests across tasks rather than averaging task shares.

37.0% of reads and 51.0% of writes. The interval $(4, 64]$ KiB contained a further 36.3% of reads and 43.0% of writes (Figure 1(b)). This mixture calls for measuring how fetching and object formation transform requests before selecting storage granularity.

Variation across tasks and time. Tasks generated 8,542–634,726 calls over capture spans of 42.7–556.1 seconds. Astropy contributed 55.7% of all requests, so the pooled distributions describe recorded traffic rather than an equally weighted typical task. The busiest 10% of one-second windows contained 47.4–83.7% of each task's calls (Figure 1(c)). Windows include idle and final partial windows; the number selected is rounded up per task. These differences favor retaining per-task mixes and timing in replay.

Measurement limits. The capture contains 872,519 successful, 253,669 error, and 12,696 unpaired events. Errors are lookup and attribute probes except for 14 `rmdir` calls; missing errno values prevent finer attribution. Unpaired events are `forget` and `destroy` records. Each task was observed once, and task-grading success was not established. The traces characterize requests reaching the passthrough file system. They do not establish remote latency, ML access patterns, cross-task sharing, sparse mutation, or BrewFS performance.

2.4 Design implications

Figure 1 establishes the operation mixes, request lengths, and per-task timing to retain in replay. The workspace and ML scenarios supply two further design targets: retaining unchanged state when attempts diverge, and serving a fixed small-file namespace without repeatedly querying the mutable metadata service. These costs arise at different interfaces. Section 3 places private state in the workspace view, fixed-record queries in the metadata representation, and transfer and verification units in the data path.

3 System Overview

3.1 File views and storage interfaces

BrewFS exposes file operations through a FUSE client backed by a metadata service and object storage. An ordinary volume keeps its namespace in mutable metadata. A workspace instead resolves private records over a pinned immutable lower. Both views locate file bytes through data mappings, allowing changes to a name or range to preserve references to the remaining data.

Figure 2 shows where the three mechanisms act. Delta-Based Workspaces compose the file view (Section 4). Packed Metadata supplies an alternative representation for a selected read-only lower (Section 5). Variable-Granularity Object Storage governs object formation and the BrewFS–RustFS layout interface (Section 6). An ordinary volume can use that interface, and a packed version can be mounted directly for read-only access.

3.2 Binding shared state

A workspace binds a fixed lower version and a private writable upper. The version may be a sealed native layer or a packed manifest covering a selected subtree. Pinning that identity gives every inherited lookup the same starting view even as other workspaces change or newer versions are published. The private upper determines which names, attributes, and byte ranges override that view.

The lower-query interface resolves names, attributes, and data locators. Native metadata answers through the catalog; Packed Metadata answers through authenticated indexes and joint pages. This interface separates workspace isolation from the preparation of a packed representation. An ML experiment can, for example, use a native writable workspace for code and outputs while mounting a separately published training dataset.

3.3 Reading and publishing data

Consider a workspace reading the first 4 KiB of an inherited file. Lookup checks private records and inheritance rules before consulting the lower. A packed lookup returns the name,

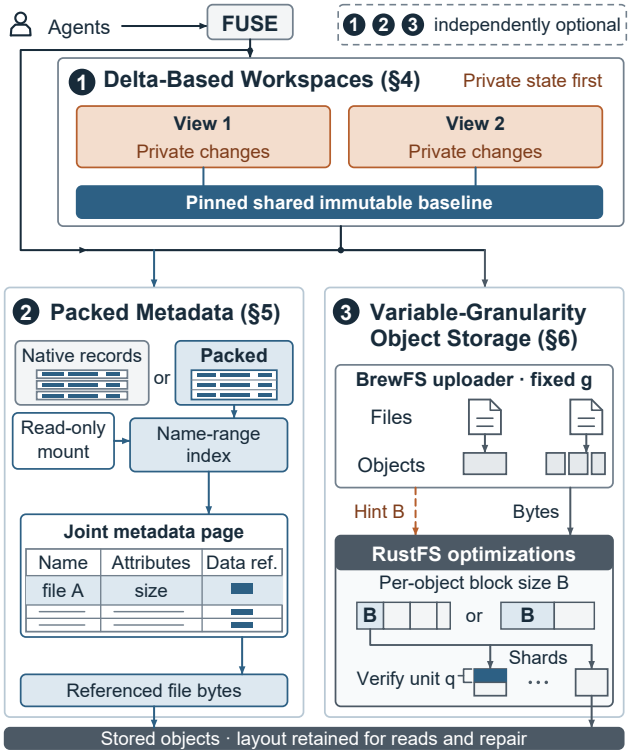


Figure 2: Three optional mechanisms in BrewFS (Sections 4–6). The left branch reaches the metadata and data paths without a workspace. Packed pages serve read-only metadata. The ordinary uploader forms objects with client block size g and can attach a B hint (dashed). Our RustFS extensions provide per-object coding-block sizing and finer shard verification units q; admitted geometry persists for reads and repair.

common attributes, and data locator from a joint page. The data path follows the locator to the stored bytes, using the representation in which those bytes were written. RustFS translates the resulting object request into shard reads under the object's persisted layout.

For new ordinary data, the client forms an object before requesting a backend coding-block size. Our RustFS extensions support per-object coding-block selection and finer shard verification, subject to backend admission and compatibility checks. Persisting the layout lets later reads and repair interpret an object independently of the configuration used for subsequent writes.

An overwrite publishes a private mapping to new data while preserving other workspaces' references. Publishing a recoverable version requires durable objects and validated metadata; retention keeps referenced objects live. Section 4 gives the update rules, and Section 7 describes writeback and recovery.

4 Delta-Based Workspaces

4.1 A shared baseline and private changes

The unit of private state determines the cost of an isolated edit. Copying a namespace at fork repeats its metadata; copying an inherited file on its first write can materialize bytes the attempt never changes. Delta-based workspaces instead retain private records for changed names, attributes, deletion markers, and data ranges over a shared immutable baseline. The baseline keeps inherited references stable, while private records describe the current visible state.

Workspace creation registers the immutable base and the private control state without copying the metadata tree. The catalog identifies the base by layer, sealed version, and root hash; a packed lower also binds its authenticated manifest. Creation validates this binding, registers ownership, and prepares the mount. The lower can use native metadata or an already published pack (Section 5).

4.2 Resolving names

Lookup first checks the private upper. An entry supplies the result; a deletion marker reports absence and stops the lookup. Otherwise, the reader consults the lower unless an opaque directory or another inheritance rule hides the name. Those rules suppress inherited entries, leaving explicit upper entries visible. Deleting an inherited name must retain its marker, since removing the upper record alone would expose the lower name again.

Directory enumeration applies the same precedence and removes duplicate names. Both the buffered and direct-I/O workspace paths preserve an inherited file's inode number when materializing private attributes or mappings. Updates through one hard-link alias therefore remain visible through the others.

Rename must change the source and destination bindings together. The direct-I/O path locks participating inodes in number order, validates the observed records and writer authority, and commits the source deletion marker, destination binding, and affected inode and parent records in one transaction. Namespace reads validate an operation-local view and retry conflicts, preventing a partially applied rename from becoming visible within an operation. Section 7 describes the buffered path and the scope of these guarantees.

4.3 Overwrites and truncation

An overwrite changes the data references for a file range. The buffered path records private extents alongside namespace and inode changes. The direct-I/O path represents each changed logical chunk with a normalized map of upper data, lower references, and zero-filled holes. Within the range a map covers, every interval names exactly one source, which

makes the visible source unambiguous. Ranges without a private override inherit the lower's bytes up to the lower-visible boundary. This boundary only shrinks as the workspace truncates inherited data.

A short overwrite replaces its interval and retains lower references elsewhere. Truncation must invalidate both inherited and previously written bytes. The direct-I/O path first commits the reduced size, a nonincreasing lower-visible boundary, and a cleanup intent. Cleanup replaces discarded spans with zeros; extension completes pending cleanup before increasing the size. For example, after an overwrite of 6–10 KiB, truncation to 7 KiB, and extension to 12 KiB, the range 7–12 KiB must read as zeros. Reducing only the inherited boundary would leave the discarded upper bytes available to a later extension.

4.4 Publication and retention

The direct-I/O path uploads an object durably before publishing its mapping. Ordinary buffered writeback also defaults to upload-before-commit, but offers an explicit commit-before-upload mode. That mode can expose mappings before their objects are remotely available. Preparing a recoverable version must therefore drain and validate referenced objects, even when local reads already return the new data.

Worker handoff requires stopping source writes and making payloads and metadata durable before publishing the version. The destination binds that version and acquires write authority; lease and generation checks must reject stale writers. Applications checkpoint execution state and reopen files separately, because a published file version carries file contents and names, not process memory or open handles.

Reclamation retains every object still referenced by a workspace or reader and distinguishes these objects from abandoned construction output. Private metadata grows with modifications and deletion markers; retained baselines and ownership records contribute shared overhead. The evaluation accounts for both live references and objects awaiting reclamation.

5 Packed Metadata

A fixed collection requires no namespace updates, yet serving it through the mutable catalog can consume the same query capacity as active workspaces. Packed Metadata uses immutability to change this service path. It organizes a directory version into authenticated object pages from which readers obtain names, common attributes, and data locators. Construction is paid per version; subsequent readers reuse the result.

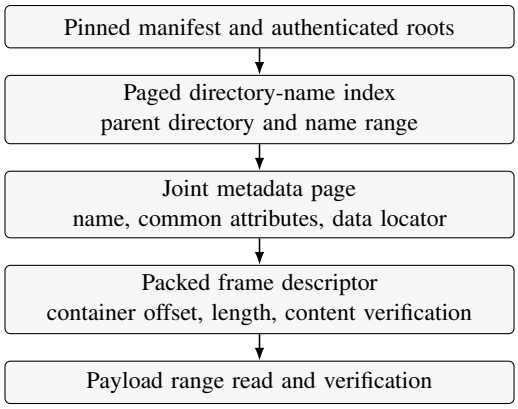


Figure 3: Packed-snapshot lookup and data access. A name lookup returns joint metadata and a data locator. A file read resolves authenticated descriptors before fetching and verifying the referenced payload range.

5.1 What is packed

Packed Metadata applies to a selected read-only directory version that has been explicitly built and published. It can serve a standalone read-only mount or supply a workspace lower. Writes in the latter case go to the private upper, leaving the published pages immutable.

A logical read group contains entries from one parent directory over a range of raw names in byte order. Its joint metadata page co-locates each entry with common attributes and data locators. A name lookup can therefore supply information needed by subsequent attribute and data operations from the same fetched page. During enumeration, a decoded page supplies successive entries in its range. Section 7 describes the caches used by individual name queries.

Raw pages are bounded at 256 KiB. A physical container may hold several groups, so the metadata access unit need not equal the stored object. Range indexes route queries to groups; index height depends on page capacity and record count rather than source-directory depth.

5.2 Querying a pinned snapshot

A reader pins a manifest and starts from its authenticated index roots. For a name lookup, the directory-name index identifies the group covering the key. The reader resolves the container and fetches the metadata page's byte range. Before decoding, it verifies the stored-byte digest; after decoding, it checks entry counts and name boundaries against the authenticated group reference. The matching record supplies the inode, common attributes, and data locator (Figure 3).

Other queries use indexes suited to their keys. Attribute queries addressed by inode use a separate index. Enumeration traverses ordered name ranges, with counted indexes supporting directory-ordinal navigation. A file read returns

inline contents or follows a locator to an authenticated descriptor and verified payload range. Packed data frames define snapshot-level access and integrity units; the backend serves them using its own persisted object layout.

Every returned record or payload must connect through authenticated references to the pinned manifest. Missing objects and failed authentication are errors, not evidence that a name is absent. Cached records remain valid for that identity after newer versions appear. A cold lookup may still require several index requests; data reads can add descriptor and payload requests. Co-location provides opportunities for reuse without fixing the number of GETs per operation.

Supported lower queries need no per-file record in the mutable database. Private-upper checks, mutations, and ownership coordination continue through the workspace and control services.

5.3 Building and publishing a version

Construction starts from a consistent source view, established by a snapshot or validation protocol. This captures one directory version before its records are reorganized. The producer groups records, uploads objects, and builds the indexed manifest. Publication installs the binding after the referenced objects are durable and valid. Retained versions and readers keep their objects live until reclamation is safe.

If no published pack exists, the first packed fork additionally waits for construction, upload, validation, and publication. A native overlay requires native version capture and binding but avoids pack construction. Later children reuse the published pack rather than rebuilding it. Query offload can help while the original database records are retained; reducing their persistent footprint also requires safe retirement, which is a separate step from the offload itself.

The relevant cost therefore spans construction, each use, retention, and reclamation. Grouping gives related queries a common page to reuse; the available caches determine whether that reuse occurs. Isolated lookups may transfer unused records. Inlining, compression, and payload co-packing introduce additional effects that must be measured separately. Long retention or rapid version churn can outweigh access savings even when queries become faster.

Section 8 varies the number of readers, query locality, and version lifetime to identify when reuse repays this preparation cost.

6 Variable-Granularity Object Storage

Object formation and physical access make different tradeoffs. Small client objects limit whole-object transfers but add requests and per-object overhead. Backend coding blocks and checksum frames determine how an object is encoded and how much shard data a partial read verifies. We co-design BrewFS and a RustFS extension to expose these decisions across the file-system–object-store boundary. The client supplies a formed object and an optional coding-block-size hint; RustFS admits and stores the effective layout.

6.1 From file ranges to objects

The ordinary data path maps file offsets to logical chunks and resolves the visible slices within each chunk. A chunk is an addressing range; slices identify the content supplying its visible intervals. The uploader partitions a slice using client block size g and addresses each block by slice identity and block index. A 10 MiB slice with $g = 4\,\text{MiB}$, for example, can produce raw blocks of 4, 4, and 2 MiB.

Compression and framing determine the finalized upload length L_o. Thus g partitions the slice, while L_o measures the bytes uploaded. Flush boundaries and partial writes affect object formation, and a tail need not occupy a full block. Readers use the same geometry to translate slice intervals into object references and offsets. Replacing one interval can leave untouched intervals referencing older objects.

Chunk size must be a nonzero multiple of g. Because readers reconstruct addresses from this geometry, changing g for existing data requires a compatible representation or migration. The implementation fixes this geometry for a compatible configuration.

6.2 Co-design with RustFS

The client knows the finalized upload length L_o after assembly, compression where applicable, and framing. It can therefore choose a hint for the object that will actually be stored. Our RustFS extension accepts a requested erasure-coding block size B, checks write eligibility and deployment compatibility, and selects the effective size. An omitted or inapplicable hint uses the default layout. Payload upload proceeds through the same interface in either case. The hint leaves client geometry g and multipart part sizes unchanged.

RustFS stores the admitted coding geometry in the object's layout metadata. Subsequent reads and repair use that stored geometry rather than re-evaluating a request hint or the current write configuration. Later write-configuration changes therefore leave existing objects readable. The client retains an object's hint across upload retries. Multipart initiation chooses the layout using the complete object's length; its parts share that decision.

The physical verification/read quantum q determines how much shard data is verified together. A separate backend policy can select smaller units for eligible new framing. Full stripes must contain a whole number of verification units per shard, with coding blocks aligned across all data shards. Hint admission and new verification framing have separate compatibility gates, and both decisions are recorded in the object layout.

g	Object count	Whole-object bytes fetched
1 MiB	64	2 MiB
4 MiB	16	4 MiB
8 MiB	8	8 MiB
16 MiB	4	16 MiB

Table 1: Arithmetic example of object formation and a 2 MiB partial read. Complete intersecting objects are fetched, with no caching or compression and ignoring framing overhead. Range reads can transfer less.

6.3 Physical work for a partial read

Range GET can retrieve part of an object when the format permits independent access. The ordinary store supports page-aligned range reads for eligible small requests; compressed blocks or whole-object fetching can require larger transfers. Packed frames provide a further decoding and integrity boundary inside snapshot objects. Their size is distinct from both g and q. Caching and concurrent-read coalescing determine which requests reach the backend.

The backend maps an object range to shard intervals and rounds each interval outward to verification-unit boundaries. A 2 KiB shard read at offset 3 KiB, for example, touches two 4 KiB units: 8 KiB plus checksums, before tails or batching. Client fetching, compression, and coding determine the shard interval. Smaller q can reduce over-read but adds checksum overhead. With unchanged data and parity shard counts, nominal redundancy remains constant; a large B does not imply a fully padded tail stripe.

6.4 Granularity tradeoffs

Table 1 illustrates the tradeoff in g for an uncompressed 64 MiB slice starting at offset zero. Increasing g reduces object count but increases transfer for a read of $[10, 12)$ MiB when complete objects must be fetched.

These layers require distinct measurements. Network read amplification compares object-response bytes with application bytes served; physical read amplification uses storage-device bytes as the numerator. Upload amplification compares uploaded bytes with application bytes written. Request count, compression work, and cache occupancy capture additional costs. Compaction, retries, and short slices can increase upload work, so smaller objects need not reduce write amplification.

Smaller verification units target the excess shard bytes introduced by integrity boundaries. Their end-to-end effect also depends on client fetching, decoding, and caching. Snapshot producers and workspace upper writers use their own object formats; Section 7 identifies the paths that issue hints and the paths that fetch whole objects.

7 Implementation

BrewFS implements the representations through buffered workspaces, direct-I/O workspaces, and standalone read-only mounts. Their cache, writeback, and recovery policies determine which operations and performance effects the evaluation can compare.

7.1 Access paths

Ordinary volumes use mutable metadata and buffered writeback. Buffered workspaces add private-state resolution over a sealed catalog or packed lower. They serialize local mutations and use version-guarded catalog transactions. Dirty buffers satisfy local reads before writeback; recovery additionally requires the referenced data to become durable. Standalone packed mounts omit the private upper and currently require zero payload-cache and prefetch budgets.

The authenticated packed reader caches index pages, per-name results including absence, and attributes. Directory enumeration retains and reuses the decoded joint page for its current group. Independent name lookups fetch and decode that page on a per-name cache miss; decoded-page reuse across those queries is not implemented.

Direct-I/O workspaces store private namespace and mapping records transactionally, implementing the rename and truncation rules in Section 4. Namespace protocols on both paths are scoped to individual operations; separate calls can observe different committed versions. Direct-I/O workspaces bypass buffered writeback and metadata time-to-live caching, with admission bounds on operation sizes, enumeration, and ownership. This path supports fresh workspaces without process-crash takeover. It authenticates an entire upper object before returning a subrange, so smaller backend verification units do not reduce the bytes this client fetches.

7.2 Publication and recovery

Publication validates durable objects before installing a lower binding and acquiring reader ownership. Recovery checks generations and leases; a manifest identity alone conveys no write authority. The connected recovery path reuses the same storage allocation and checks worker identity and scoped metadata authority. Catalog operations provide quiescence, verification, retention, and atomic version transitions. Cross-worker handoff requires runtime orchestration to connect them to session checkpoints and tool reopening.

7.3 Layout support

The ordinary uploader defaults to $g = 4$ MiB within 64 MiB chunks. Readers use the same geometry; incompatible configurations are rejected. The RustFS extension accepts ex-

plicit coding-block-size hints of 64 KiB, 256 KiB, 1 MiB, and 4 MiB. Hint admission and new verification framing have separate rollout gates requiring deployment-wide compatibility confirmation. The ordinary uploader passes hints; direct workspace writes and the snapshot producer use defaults. Existing objects retain their stored layout after configuration changes.

The current automatic policy omits hints for $L_o \leq 1\,\text{MiB}$ and requests $B = 1\,\text{MiB}$ otherwise. RustFS's default coding-block size is also 1 MiB, so this policy can leave geometry unchanged. Enabling hints also changes the ordinary client's fetch and cache policy. The evaluation therefore records actual (g, L_o, B, q) and holds the client policy fixed when isolating the effect of a backend layout change.

8 Evaluation Plan

Formal performance and end-to-end recovery experiments remain to be conducted. The completed characterization in Section 2.3 supplies request mixes and timing. The following five questions test the expected benefit and cost of each mechanism, then examine workspace handoff and concurrent execution.

8.1 Systems, inputs, and accounting

JuiceFS is the external distributed-file-system baseline [8]. Internal comparisons use ordinary volumes, native-metadata workspaces, and packed-metadata workspaces. Mechanism ablations keep the execution engine, cache policy, and durability semantics fixed. Comparisons between complete access paths are reported separately. Matched resource pools use independent metadata and storage services for concurrent comparisons; runs sharing a service execute sequentially.

Inputs include coding and build/test traces, a million-file collection with 4 KiB payloads, and fixed datasets shared by concurrent readers. The experiment repository specifies generation, repetition, placement, software revisions, and configuration. Replay retains per-task request mixes and timing; actual agent continuation is evaluated separately.

Accounting covers preparation, execution, validation, retention, and reclamation, including failed attempts. Shared preparation is charged once per baseline, and object residence continues until reclamation. CPU time, transferred bytes, and monetary cost retain separate units. Batch duration includes queueing and is measured directly.

8.2 Overlay fork and retained branches

RQ1. How much state and work does overlay fork save without packing metadata? Many children start from one revision and modify a small part of its tree. A native-metadata workspace is compared with an eagerly materialized private view in the same engine, preserving payload sharing and the API. An ordinary copied volume provides a separate system-level comparison.

Fan-out and star, branching, and chained histories expose preparation sharing and the cost of forking modified parents at equal fork budgets. We measure baseline capture, creation to first correct operation, first durable edit, save latency, private metadata, and uploaded bytes. Readback checks inheritance and parent/sibling isolation. Retaining, reopening, and deleting branches measures accumulated state and actual reclamation, including outstanding cleanup and admission failures.

8.3 Read-only metadata service

RQ2. When does packing a fixed small-file collection reduce database load enough to repay construction and publication? Readers repeatedly access a fixed collection while other workspaces update the same metadata service. The representation ablation changes native versus packed metadata within one engine, holding payload layout and caches fixed. Separate comparisons allow each complete format to use its own data layout. Standalone read-only mounts isolate metadata service from private-upper checks.

Preparation includes native version capture and binding, plus pack construction, upload, validation, and publication where required. First and subsequent forks distinguish preparation from reuse. Varying reader count, version churn, and retention identifies break-even regions for latency, CPU, transfers, and residence cost. Database query offload and persistent database size are measured separately.

Positive and negative lookups, inode attributes, neighboring names, enumeration followed by attributes, and lookup followed by reads expose different locality. Cold, index-warm, and metadata-warm runs use declared cache budgets. Counters separate lower-database, private-upper, and control traffic from index, joint-page, descriptor, and payload requests. Latency, database CPU and memory, transfers, decoding, and verification show where work moves. A fixed-rate mutable-workspace probe measures interference through queueing, deadline completion, and tail latency.

An ML case separates dataset discovery from sample loading: fresh enumeration versus sample-list reuse, followed by ordered or shuffled access, with outputs outside the fixed dataset. Larger files test the role of per-file metadata. Compatible Nydus or SOCI readers provide read-only layout comparisons at matched content, integrity, cache, and payload settings; an immutable sorted-KV control isolates joint-page organization. Each comparison includes construction and version updates.

8.4 Client–RustFS layout co-design

RQ3. How do client and backend granularity affect physical work, write completion, and repair? Small reads within larger objects expose excess fetching and verification; scans expose the request overhead of smaller units. Identical writes compare client sizes of 256 KiB, 1 MiB, and 4 MiB at fixed backend geometry. Fixed finalized objects and S3 requests then compare RustFS's default layout with admitted coding-block-size hints. Verification policies are varied separately at a fixed effective B. Range and whole-object fetching have matched cache, coalescing, and prefetch policies.

Boundary-crossing reads, shared and disjoint keys, overwrites, and supported degraded reads test the tradeoffs. Object counts, finalized lengths, HTTP and device bytes, checksum and encoding CPU, durable-write latency, and repair traffic explain the results. The recorded (g, L_o, B, q) distinguishes requested hints from actual interventions; only a changed effective layout establishes a backend-layout comparison. Physical-read claims require device counters in addition to HTTP bytes.

8.5 Migration and recovery

RQ4. How much work does migration require before and after publication? An agent resumes on another worker while neighboring tasks continue. Paths that reopen an existing workspace enter paired stay/move runs; fresh-only paths enter creation experiments. Host identities establish the move, and JuiceFS uses its native directory reopen operation.

Handoff occurs at a completed-tool boundary, with either a published version or pending modifications. Interruption includes queueing, draining, capture, required packing, publication, and authority transfer. Cold and warm targets measure the first correct operation, continuation cost, transfers, and neighboring tasks' completion and tail latency.

File-state tests check names and bytes. Actual agent tests export and import session history and remaining budget, require a subsequent valid tool result, and grade final task completion. Process memory and open handles are outside file-version publication. Failures around publication, disconnected source coordination, and target failure test recovery. Independent readback distinguishes storage-enforced stale-writer rejection from process termination or application checks.

8.6 Concurrent tasks and analysis

RQ5. Do component savings increase useful capacity at fixed resources? A fixed assignment set runs at increasing concurrency under a common deadline, including queueing. A separate scale study increases workers and assigned work against fixed backend capacity. Placement and replay-engine CPU limits are recorded.

Internal ablations change one supported mechanism; composition is compared with its best supported proper subset. Paired blocks report raw outcomes and ranges. Stronger inference uses a sample size chosen from a separate variance pilot; requests and sibling tasks are not independent replicates. Deadline completion includes failures in its denominator. Complete-success makespan is reported when every assignment succeeds, alongside failed outcomes and measurement gaps.

9 Related Work

Private workspaces. OverlayFS combines upper and lower trees through copy-up and deletion markers [13]. BranchFS adds a fork, explore, and commit lifecycle; its evaluated prototype copies an inherited file on first modification [22]. BrewFS represents private names, attributes, and file ranges over an immutable baseline. Unchanged ranges retain their original references, and the lower-query interface accepts native or packed metadata. The distinction is the state copied and resolved on each operation. YoloFS provides staging, reversible effects, and permissions [29]; AgentFS records files, key-value state, and tool calls in SQLite [4]. These systems address execution control and observability, which can use the workspace and storage mechanisms described here.

Mutable metadata services. TableFS and IndexFS organize metadata and small files in tables; IndexFS also partitions directories and supports caching and bulk insertion [18, 19]. LocoFS and InfiniFS distribute metadata responsibilities [12,14], while Mantle and SwitchFS address hierarchical metadata and asynchronous directory updates [11,26]. These designs improve mutable metadata service. BrewFS instead specializes the service path of a selected read-only version: supported lower queries fetch object-backed pages. Private updates and lifetime control remain in the mutable service. The resulting tradeoff is database query work displaced versus page access and version-publication cost.

Private namespaces and immutable logs. DeltaFS publishes private-namespace mutation logs as SSTables for subsequent jobs to select and merge. Job-local servers and compaction reduce dependence on a globally synchronized metadata service [28]. BrewFS maintains a current-value private upper over a pinned lower. A separate publication step can organize the lower by directory and name range, placing attributes and data locators beside names. Native-metadata workspaces omit that construction. The two designs therefore differ in how private updates are represented and when an immutable query representation is built.

Read-only distribution and indexing. CernVM-FS uses nested SQLite catalogs to publish content-addressed trees and fetch metadata and data on demand [2]. It directly demonstrates the value of serving immutable collections outside a mutable metadata service. OCI specifies layered archives [16]; SOCI indexes image data for lazy access [20]. Nydus separates filesystem metadata from chunked data and supports caching and integrity verification [15]. Slacker and DADI likewise use demand-driven access for image distribution [5, 10]. These systems provide close precedents for immutable, remotely accessed file representations.

CoFS specializes a fixed image namespace with minimal perfect hashing and a dense metadata array. In-kernel lookup avoids userspace interposition on cached-data paths [23]. BrewFS organizes directory names, attributes, and data locators into bounded joint pages reached through authenticated range indexes. A page is the remote metadata access unit, independently of its physical container. This organization targets repeated queries over a published directory version alongside mutable agent workspaces. Compared with image indexes and catalogs, the relevant differences are the records fetched per operation, their reuse across queries, and the cost of constructing and retaining each version. The comparison concerns access during execution as well as initial materialization.

Object granularity. Ceph and Tectonic separate metadata and storage services [17, 25]. Azure's erasure coding reduces repair traffic; Morph studies redundancy transitions over a file's lifetime; Okapi separates striping from redundancy grouping [1,6,9]. Our co-design with RustFS connects filesystem object formation to per-object coding geometry. The client can request a coding-block size after forming an object. RustFS validates the hint, selects verification framing under a separate policy, and persists the effective layout for reads and repair. The interface exposes these access units while retaining the backend's erasure code and its control over admissible layouts.

10 Conclusion

BrewFS separates the representation of private workspace changes from the service of shared metadata and the layout of stored bytes. Range mappings preserve unchanged data, authenticated joint pages serve published read-only metadata outside the mutable database, and a co-designed RustFS interface carries object-layout decisions through upload, read, and repair. These mechanisms remain independently selectable, with preparation and access costs appropriate to different workloads. The implementation and eight-run request characterization provide the basis for evaluating their effects on workspace creation, repeated access, and worker handoff.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC application software and conditions databases using the CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[4] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[5] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[6] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[8] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[9] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[10] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[11] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[12] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[13] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[14] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[15] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[16] Open Container Initiative. Image layer filesystem changeset. OCI Image Format Specification. Accessed 2026-10-26.

[17] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[18] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[19] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[20] SOCI Snapshotter Contributors. SOCI snapshotter. Official project documentation. Accessed 2026-10-26.

[21] Torchvision Contributors. DatasetFolder and ImageFolder. Torchvision documentation and implementation. Accessed 2026-10-26.

[22] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[23] Li Wang, Jinxu Du, Yang Yang, Qingbo Wu, Tao Liu, and Haoze Wu. CoFS: A filesystem for fast container startup. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 415–423. USENIX Association, 2026.

[24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[25] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[26] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous

metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[28] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[29] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.